CHAPTER 25

Guardrails, Validation and Prompt Debugging

In this chapter

Introduction: The Guardrail That Blocked Five Thousand Customers . . . 3

25.1 Guardrails Are Layers, and Each Layer Has a Job . . . 4

25.2 A Guardrail Is a Classifier, and Classifiers Meet Base Rates . . . 5

25.3 The Refusal Contract, in Full . . . 5

25.4 Falling Back: Three Approaches . . . 7

25.5 What the Fallback Response Contains . . . 8

25.6 Fallback Credit: Not Paying Twice . . . 9

25.7 Validation Has Three Independent Senses . . . 10

25.8 Debugging a Prompt . . . 11

25.9 Visual Analogies . . . 11

25.10 Architecture: The Guarded Call . . . 12

25.11 Examples . . . 13

25.12 Code Walkthrough: The Reader That Assumes Nothing . . . 13

25.13 Mini Project: The Guardrail Ladder and the Fallback State Machine . . . 14

25.14 Panel A — What a 99/99 Guardrail Does to Real Users . . . 15

25.15 Panel B — Seven Refusals, One Reader . . . 15

25.16 Panel C — Eight Fallback Rejections, No API Key . . . 16

25.17 Panel D — The Retry, and What the Credit Is Worth . . . 16

25.18 Panel E — The Tokens You Have Already Shown Someone . . . 17

25.19 Panel F — Ablation, One Part at a Time . . . 17

25.20 Evaluation and Deployment . . . 18

25.21 Hands-on Labs . . . 19

25.22 Lab Solutions . . . 19

25.23 Best Practices . . . 19

25.24 Common Mistakes . . . 21

25.25 Performance Considerations . . . 24

25.26 Security Considerations . . . 24

25.27 Real Production Examples . . . 25

25.28 Certification Notes . . . 25

25.29 Twenty Practice Questions 26
25.30 Answers and Explanations 27
25.31 Ten Practical Exercises 28
25.32 Five Challenge Labs 28
25.33 Ten Interview Questions 28
25.34 Summary 29
25.35 Glossary 30
25.36 Further Reading 31

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Design guardrails as **layers** at four points — input, model, output and action — and say what each layer can and cannot do.
2. Compute what a classifier guardrail does to real users at a real base rate, and reject one that cannot work before you build it.
3. Read the full refusal contract: HTTP 200, possibly empty `content`, `stop_details` with `type`, `category` and `explanation`, and the four documented categories.
4. Explain exactly when a refusal is billed and when it is not, and why an error-rate monitor never sees one.
5. Choose among the three documented fallback approaches — server-side `fallbacks`, the SDK middleware, and a manual retry — and configure each correctly.
6. Redeem a **fallback credit token**, pick the right retry body from `fallback_has_prefill_claim`, and walk the documented rejection ladder.
7. Validate an output in three independent senses — schema, semantics and authority — and know which of the three a grammar gives you free.
8. Debug a prompt by **ablation with intervals** rather than by editing, and never change two things at once.

NOTE

Prerequisites

Chapter 8 for the layered safety architecture, for Cohen's kappa, and above all for the base rate destroying accuracy as a metric — that arithmetic is the spine of this chapter's first half. **Chapter 10** for the capability triage that decides whether a failure is a prompting problem at all. **Chapter 16** for `stop_reason` as the control-flow branch and for the refusal-as-HTTP-200. **Chapter 17** for streaming, for the `fallback` content block, and for fixture provenance. **Chapter 19** for cache multipliers and what a token costs. **Chapter 20** for a prompt as parts with a kind and a stability. **Chapter 23** for schema validation. **Chapter 24** for paired intervals and for the fact that an ablation without power is an opinion.

This chapter closes Part 4. **Prompt injection in full** — the attack, not the guardrail — is **Chapter 58**. Observability and tracing in production are **Chapter 57**. Reliability failure modes in the round are **Chapter 56**.

Introduction: The Guardrail That Blocked Five Thousand Customers

A company shipped a jailbreak classifier in front of its assistant. The classifier was good: 99 per cent sensitivity and 99 per cent specificity, measured on a balanced test set, signed off by two teams. The dashboard reported 99 per cent accuracy and everybody agreed that was excellent.

Two weeks later support was drowning. The genuine attack rate on that product was about **0.2 per cent** of requests, and Section 25.14 reproduces what that does. On a million requests the classifier catches 1,980 genuine attacks, misses 20, and blocks **9,980 legitimate users**. Precision is **16.56 per cent**: **five**

real customers are turned away for every attack stopped. Accuracy is still 99 per cent. It was 99 per cent the whole time.

Chapter 8 made this point about safety metrics in the abstract. Here it is the design decision. To reach even 50 per cent precision at that base rate the layer would need a specificity of **0.998016** — a false-positive rate five times better than the one everybody was so pleased with — and nobody computed that number, because computing it takes one line and asking for it takes courage.

That is the first of this chapter's four failures. The second is quieter and more common. The same company's alerting was built on HTTP error rates, and across the six BUILT refusal fixtures in Section 25.15 an error-rate monitor sees **zero of six**. A refusal is a successful response. Every dashboard built on 4xx and 5xx status codes is structurally blind to the single most important thing a safety system does.

The third is the code. `response.content[0].text` — Chapter 16's named bug, still with us — raises on **six of seven** refusal payloads, because on a pre-output refusal `content` is not merely the wrong block, it is the **empty array**.

And the fourth is the one that wastes the most engineering time. A team debugging a prompt with nine parts removed two of them together, saw accuracy fall 3.45 points, and concluded that the anti-laziness instruction had been holding things up. Section 25.19 runs the ablation properly, one part at a time with paired intervals: **four of the nine parts have an established effect and five cannot be shown to do anything at all**, and the anti-laziness instruction is established as **harmful, at −2.65 points** — exactly the over-triggering the documentation warns about, and exactly what the two-at-once experiment hid.

Part 4 began with principles, moved through the system prompt, XML, schemas and evaluation. It ends here, with what to do when the thing you built says no.

25.1 Guardrails Are Layers, and Each Layer Has a Job

Chapter 8 established the layered architecture. This section makes each layer's job precise, because most guardrail failures are a layer doing another layer's work.

Layer	Runs	Can	Cannot
Input	before the model call	reject known-bad input, strip PII, rate-limit an abuser	judge what the model will do with ambiguous input
Model	inside the call	refuse, per Anthropic's own classifiers and the constitution	be configured off, or be relied on alone
Output	after the response	check schema, check semantics, check policy	undo tokens already streamed to a user
Action	before the effect	authorise, sign, rate-limit, require a human	know what the model meant

Two of these are yours, one is the platform's, and one — the action layer — is the only one that can actually prevent harm, because it is the only one between the model and the world.

KEY TAKEAWAYS

The layer that matters is the last one

Chapter 21's thread, unchanged: **the model proposes, your system disposes.** An input filter reduces load. An output check reduces embarrassment. Only the authorisation check in front of the effect prevents the refund, the deletion or the email. Build that one first and the others become optimisations.

25.2 A Guardrail Is a Classifier, and Classifiers Meet Base Rates

Every filter you write is a binary classifier with a sensitivity and a specificity, whether or not you have measured them. The consequences depend almost entirely on a number that has nothing to do with the classifier: how often the bad thing actually happens.

Section 25.14 runs one classifier — 99 per cent sensitivity, 99 per cent specificity — at three base rates.

Base rate	Accuracy	Recall	Precision	Legitimate users blocked per genuine catch
5.00%	99.00%	99.0%	83.90%	0.19
1.00%	99.00%	99.0%	50.00%	1.00
0.20%	99.00%	99.0%	**16.56%**	**5.04**

The accuracy column never moves. That is the whole lesson, and it is Chapter 8's, restated as an engineering decision rather than an observation about benchmarks.

The useful design tool is to run the calculation backwards. Given a base rate and an acceptable precision, what specificity would the layer need?

> specificity ≥ 1 − [R·b(1−p)] / [p(1−b)]

At b = 0.002, R = 0.99 and a modest p = 0.50, that is **0.998016**. If your classifier cannot demonstrate a false-positive rate below 0.2 per cent — with an interval, on a realistically imbalanced sample — the layer will block more customers than attackers and you have learned that before writing it.

Stacking does not fix it. Two independent layers at 90 per cent sensitivity and 99.5 per cent specificity give a combined recall of 99.0 per cent — as good as the single strong layer — and a precision of **16.59 per cent**, with **9,955** legitimate requests blocked. Series composition multiplies the misses, which is what you wanted, and it also compounds the false positives, which is not. Layers in series buy recall and spend precision.

What actually helps is changing the consequence rather than the classifier: route flagged requests to a slower, more careful path instead of blocking them; ask a clarifying question; or move the decision to the action layer, where the base rate of "this specific request wants to move money" is far higher and the same classifier suddenly has usable precision.

25.3 The Refusal Contract, in Full

Chapter 16 introduced `refusal` as one of seven `stop_reason` values and as an HTTP 200. The dedicated documentation page adds enough detail to change how you write the handler.

A refusal is a successful response whose `stop_reason` is `"refusal"`. On Claude Fable 5 the safety classifiers can decline a request; when they do, `content` may be **empty**, and `stop_details` carries three fields:

```
{
  "content": [],
  "stop_reason": "refusal",
  "stop_details": {
    "type": "refusal",
    "category": "cyber",
    "explanation": "This request was declined because it could "
                   "enable cyber harm."
  },
  "usage": {"input_tokens": 412, "output_tokens": 0}
}
```

`category` names the policy area. Four are documented:

Category	What it means
`cyber`	could enable cyber harm, such as malware or exploit development — *"benign cybersecurity work can also trigger this category"*
`bio`	could enable biological harm — *"beneficial life sciences work can also trigger this category"*
`frontier_llm`	could assist the development of competing AI models, restricted under Anthropic's commercial terms — *"benign machine learning work can also trigger this category"*
`reasoning_extraction`	asks the model to reproduce its internal reasoning in the response text; use adaptive thinking to get reasoning in a structured form instead

Each of the first three carries an explicit acknowledgement that legitimate work trips it. That is a precision statement about somebody else's classifier, and it deserves the same reading as Section 25.2 gives your own: the false positives are not hypothetical, they are documented.

`explanation` is human-readable and *"the text is not stable, so display it rather than parse it."*

Both fields are `null` when the refusal maps to no named category, and the documentation is emphatic: *"That null is a normal, permanent value, not a placeholder."* On a Message Batch result, `stop_details` may be `null` altogether. Hence the documented rule, which is also Chapter 16's rule with a new reason behind it: **branch on** `stop_reason`**, not on** `stop_details` **or** `content`**.**

WARNING

Refusals are billed differently depending on when they arrive

"You are not billed for a refusal that arrives before any output. `content` *is empty, token counts appear in* `usage` *but are not charged, and the request does not count against rate limits. A mid-stream refusal bills the input tokens and the output already streamed at normal rates."*

Two consequences. First, a `usage` object with non-zero `input_tokens` does not imply a charge, so a cost model that sums `usage` across all responses will overstate your bill by the pre-output refusals. Second, and less comfortable: **the refusal that costs you nothing is the one that produced nothing, and the refusal that produced half an answer and showed it to your user is the one you pay for.**

25.4 Falling Back: Three Approaches

A refused request will usually be answered by a different model. The documentation gives three ways to arrange that, and picks between them by where you are running and how much control you need.

Situation	Approach
Claude API or Claude Platform on AWS, simplest setup	**server-side fallback** — one request, one response
Any platform, with the TypeScript, Python, Go, Java or C# SDK	**the SDK middleware** — configure once on the client
Ruby, PHP, raw HTTP, or custom retry logic	**manual retry with fallback credit**

Server-side fallback and the SDK middleware apply fallback credit for you; the credit page matters only when you build the retry yourself.

Server-side fallback takes a `fallbacks` list and the beta header `server-side-fallback-2026-06-01`:

```
client.beta.messages.create(
    model="claude-fable-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Hello, Claude"}],
    fallbacks=[{"model": "claude-opus-4-8"}],
    betas=["server-side-fallback-2026-06-01"],
)
```

The rules are exact and Section 25.16 reproduces all eight rejections offline. Up to **three** entries, tried in order, each **distinct** from the others and from the requested model. Each must be a **permitted target**, published as `allowed_fallback_models` on the model's Models API entry when the beta header is set. Each may override `max_tokens` and `thinking` for that attempt only. **The request must be valid as a direct request to every model named** — if a fallback does not support a feature you use, the whole request is rejected up front. It is rejected on the **Message Batches API** and is unavailable on **Amazon Bedrock, Google Cloud and Microsoft Foundry**. And the beta header must carry **exactly** the date `2026-06-01`; any other `server-side-fallback-*` value gets a 400.

The most important sentence in the section is about what does *not* trigger it: **"Only a safety classifier decline triggers the fallback. A rate limit, overload, or server error on the requested model is**

returned to you as-is." Fallback is not a resilience feature. Chapter 19's retry discipline is still entirely your problem.

25.5 What the Fallback Response Contains

The response looks ordinary with two additions, and both are things your parser must already tolerate.

The top-level `model` field reports the model that produced the returned message. A `fallback` **content block** marks each handoff:

```
{"type": "fallback",
 "from": {"model": "claude-fable-5"},
 "to":   {"model": "claude-opus-4-8"}}
```

Chapter 17 recorded this block arriving in a stream as a `content_block_start` / `content_block_stop` pair **with no deltas**; here is what is inside it. `to.model` is always the resolved ID of the model that continues.

And `usage.iterations` records every attempt: a model that declined appears as an ordinary `message` entry, the model that served the turn as a `fallback_message` entry. If every model declines, you get the last one's refusal with a `message` entry for each earlier hop.

```
   REQUEST  model=claude-fable-5  fallbacks=[claude-opus-4-8]
            beta: server-side-fallback-2026-06-01
                              |
                    ┌─────────▼──────────┐
                    |  claude-fable-5    |  safety classifier declines
                    └─────────┬──────────┘  usage.iterations[0]
                              |             type "message", out 0
                    ┌─────────▼──────────┐
                    |  claude-opus-4-8   |  answers
                    └─────────┬──────────┘  usage.iterations[1]
                              |             type "fallback_message"
                              ▼
   RESPONSE  model: "claude-opus-4-8"        <-- the SERVING model
             content: [
               {"type": "fallback",
                "from": {"model": "claude-fable-5"},
                "to":   {"model": "claude-opus-4-8"}},
               {"type": "text", "text": "..."}
             ]
             stop_reason: "end_turn"
   Streaming, decline BEFORE output : message_start already names the
     fallback model; TTFB includes the declined attempt.
   Streaming, decline MID-output    : the open block closes, the
     fallback block marks the boundary, and the fallback model
     CONTINUES from the partial text.
   Non-streaming, decline mid-output: the partial output is OMITTED
     and the fallback answers from scratch -- but the declined
     attempt's output tokens still appear in usage.iterations.
```

That last asymmetry is worth pausing on. **The same decline produces different content depending on whether you streamed it.** Streaming preserves the partial text and continues from it; non-streaming discards it and starts over, while still billing what was generated. If you are comparing a streaming and a

non-streaming path in the same product, they are not equivalent under refusal.

Echoing the turn back has its own rules, and they are the strictest in the chapter. Keep the `fallback` block **exactly where it appeared** — the API uses its position to validate the surrounding thinking blocks, and a request that echoes thinking from both sides of the boundary is rejected if the block is moved or omitted. Keep `text`, and keep everything after the final `fallback` block. **Drop** `thinking`, `redacted_thinking`, `connector_text` and client-side `tool_use` that appear *before* the final `fallback` block. Keep `server_tool_use` when it is paired with its result; drop it when it is not.

25.6 Fallback Credit: Not Paying Twice

Prompt caches are per model. Chapter 11 and Chapter 20 both leaned on that fact without needing to say it out loud; here it has a price tag. When Fable 5 declines and you retry on Opus 4.8, the prefix that was warm on one model is cold on the other, so the entire prefix becomes a cache **write** at 1.25× instead of a **read** at 0.1×.

> DEFINITION
>
> **Fallback credit**
>
> The refusal carries an opaque `fallback_credit_token`. Echo it on the retry with the beta header `fallback-credit-2026-06-01` and *"the retry is billed as though the conversation had been on the new model all along."* Server-side fallback and the SDK middleware apply it automatically; you need it only when you build the retry yourself.

The arithmetic is a constant, because the cache multipliers are constants (Chapter 19): a write at 1.25× against a read at 0.1× is **12.5×**, at every prefix size. Section 25.17 prices it on Opus 4.8 at $5 per MTok input:

Prefix	Without the credit	With it	Ratio
4,000 tokens	$0.0250	$0.0020	12.5×
40,000 tokens	$0.2500	$0.0200	12.5×
200,000 tokens	$1.2500	$0.1000	12.5×

Two fields arrive on the refusal's `stop_details`: `fallback_credit_token`, and a Boolean `fallback_has_prefill_claim` which chooses the retry body.

`fallback_has_prefill_claim`	Retry body
`true`	the refused request body unchanged, **plus one appended assistant message** echoing the refused response's `content` — the retry model continues from where the refused model stopped, and completed server tool calls are **not re-executed**
`false`	the refused request body, unchanged

Both fields are `null` when no credit is available.

The appended assistant message deserves a note, because Chapter 16 recorded that prefilling is gone on Claude 4.6 and later. This is not user prefilling; it is a server-validated continuation claim, redeemable only against a token the server issued. The validator is fussy in one documented way: it **rejects trailing whitespace on a final text block**, so the echoed content must be right-stripped.

And when a retry is rejected there is a documented **rejection ladder**, which Section 25.17 walks:

1. **Continuation rejected** → resend the unchanged body, still with the token.
2. **Token rejected** (a 400 naming `fallback_credit_token`) → retry without the token. The credit is forfeited; the retry goes through.

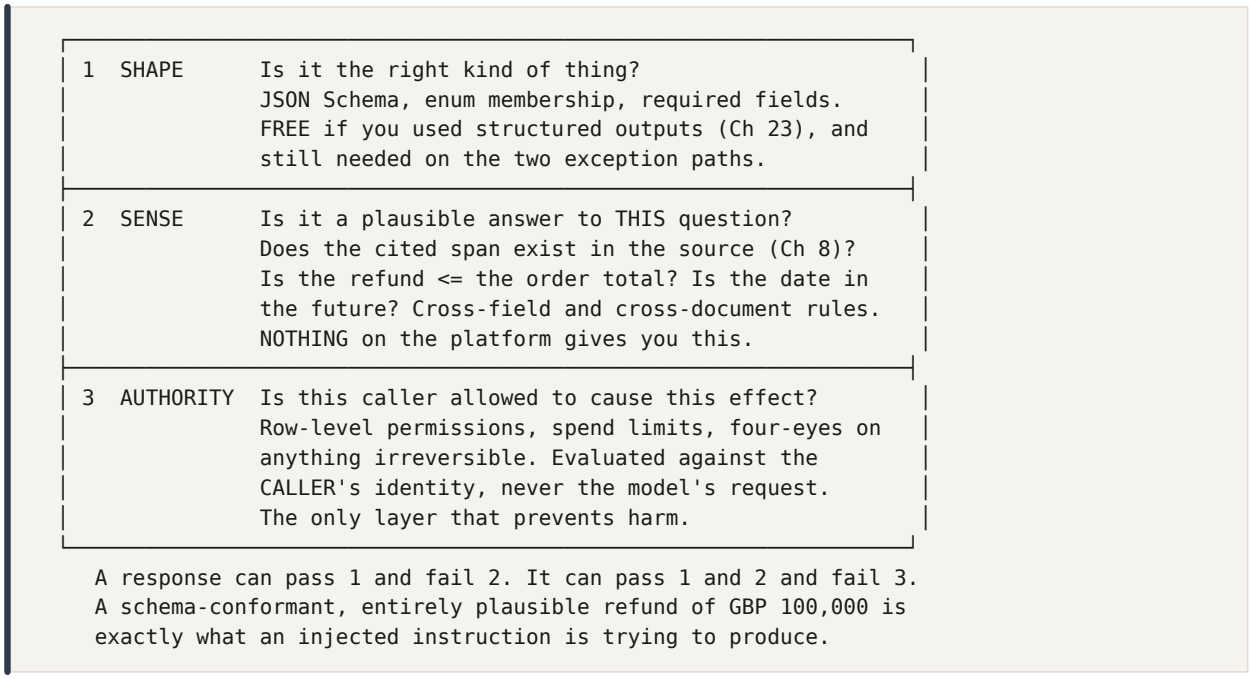
With one exception that is a genuine trap: *"If the refused request executed server tools, a tokenless retry re-runs and re-bills those tools. In that case, surface the 400 error to your caller instead of falling through to a tokenless retry."* A generic three-rung retry helper will cheerfully re-run a web search and a code execution container and bill you for both.

`"redemption temporarily unavailable"` is transient, and the token has a **five-minute** window. Verification is arithmetic: compared with the same request without the token, `cache_creation_input_tokens` is **lower** and `cache_read_input_tokens` is **higher by the same amount**. A shift of exactly zero means the token was honoured and there was nothing to reprice.

Fallback credit is available on the Claude API, Claude Platform on AWS, **Amazon Bedrock, Google Cloud and Microsoft Foundry** — a wider set than server-side fallback, which is API and AWS only. Credit tokens returned in Message Batch results **cannot be redeemed**.

25.7 Validation Has Three Independent Senses

Chapter 23 gave you schema conformance and was careful to say it is not correctness. Production needs three checks and they fail independently.



The failure to design against is a system that has built check 1 beautifully, has check 2 scattered across three files as ad-hoc `if` statements, and has check 3 nowhere, because the model "would not do that". Chapter 58 is about the attacker who makes it do that; the defence is architectural and is built here.

25.8 Debugging a Prompt

When the evaluation goes red, there is a method, and it is boring on purpose.

Step 0 — Triage before you edit. Chapter 10's four questions: is this missing *information*, *exact execution*, *external state*, or *checkability*? Prompting is the fall-through, not the first move. The documentation says it plainly: not every failing eval is a prompting problem.

Step 1 — Check the control flow before the content. Read `stop_reason` across the failures. A cluster of `max_tokens` is not a prompt problem. A cluster of `refusal` is not a prompt problem either — it is a fallback problem, and Section 25.15 shows how invisible it is if you are watching error rates.

Step 2 — Check the configuration before the words. Chapter 18's thinking settings, Chapter 15's effort level, Chapter 23's schema, the model ID. All of these are part of the prompt in the sense that matters — they are rendered into it and they invalidate the cache — and all of them are easier to change by accident than a paragraph is.

Step 3 — Ablate, one part at a time, with intervals. Chapter 20 said a prompt has parts. Debugging is therefore a leave-one-out experiment over those parts, and Chapter 24 says an experiment without power is an opinion. Section 25.19 runs it on nine parts with 2,000 paired cases each.

Step 4 — Write down what you tried. Chapter 24's winner's curse applies to debugging too: if you try eleven edits and keep the one that helped, you have selected on noise unless you confirmed it.

> COMMON MISTAKE
>
> ### Changing two things at once
>
> **Symptom.** A confident, wrong causal story. "Removing the anti-laziness instruction made it worse."
>
> **Cause.** Two edits, one measurement. Section 25.19: dropping the anti-laziness instruction alone is **+2.20 points**; dropping the examples alone is **−4.45**; dropping both reads **−3.45**, and the improvement is invisible inside the loss.
>
> **Fix.** One change per measurement, always. It is slower and it is the only thing that works.
>
> **Detection.** If your change log has entries with the word "and" in them, the effects are not attributable.

25.9 Visual Analogies

VISUAL ANALOGY

The metal detector at a small airport

Set it sensitive enough to catch every weapon and you will be patting down grandmothers all day, because almost nobody carrying anything is carrying a weapon. The detector's accuracy is not the problem; the population is.

Where it breaks. An airport can afford to be annoying because passengers cannot leave. Your users can. A guardrail with 16.56 per cent precision does not merely inconvenience — it is a churn mechanism with a security justification.

VISUAL ANALOGY

A relay race with a dropped baton

Server-side fallback is a second runner already on the track. When the first declines, the baton passes mid-stride: on a stream the second runner continues from where the first stopped; on a non-streaming request the race is restarted and only the finish time of the second runner is reported, though you still pay for the first one's legs.

Where it breaks. In athletics the baton is the same object. Here, only the partial **text** is handed over; thinking blocks and client tool calls from before the boundary are dropped when you echo the turn. The `fallback` block is the marker that tells the API where the handoff was, which is why moving it is a 400.

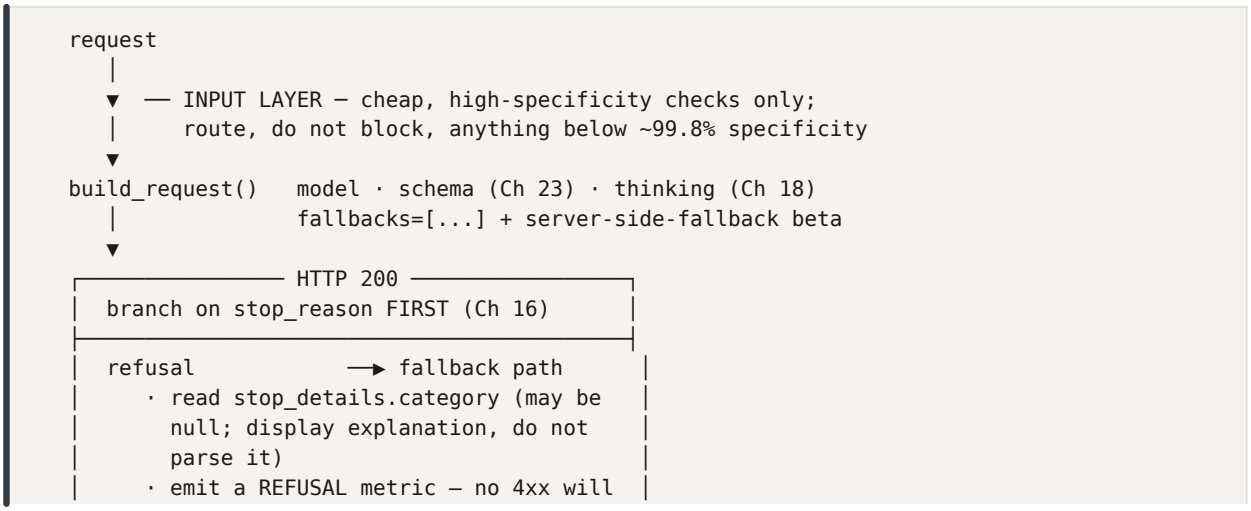
VISUAL ANALOGY

Two receipts for the same seat

Fallback credit is the airline reissuing a ticket rather than selling you a second one. The seat is the cached prefix; the credit token is proof you already paid for it on the other aircraft.

Where it breaks. The reissue expires in five minutes, cannot be used on a charter flight (a batch), and if the first aircraft already served you a meal — a server tool that ran — abandoning the ticket entirely means paying for the meal twice. That is why the documented advice is to surface the error rather than fall through to a tokenless retry.

25.10 Architecture: The Guarded Call



```
    │       ever fire for this                    │
    │     · retry on a DIFFERENT model, with the   │
    │       credit token if one was issued         │
    │     · discard any partial output             │
    │  max_tokens        ──► retry, higher cap     │
    │  tool_use          ──► the loop (Ch 31)      │
    │  pause_turn        ──► continue              │
    │  end_turn          ──► validate              │
    └──────────────────────┬──────────────────────┘
                           ▼
              1 SHAPE — schema (free under Ch 23)
              2 SENSE — cross-field and source checks
              3 AUTHORITY — against the CALLER's identity
                        │
                        ▼
                    effect
```

Three rules the picture encodes. **Fallback is a property of the request, not of ambient state** — the documentation names shared flags and cached config as a way to silently lose the protection. **Sub-agent calls need their own fallback**, because `fallbacks` does not propagate into model calls made inside tool execution. And **retries are budgeted per request, not per turn**, since one turn can produce several refusals across an agent and its sub-agents.

25.11 Examples

Reading a refusal without assuming anything.

```
if resp.stop_reason == "refusal":
    d = resp.stop_details or {}          # may be None on a batch result
    category = d.get("category")         # may be permanently None
    metrics.increment("llm.refusal", tag=category or "unnamed")
    partial = [b for b in resp.content if b["type"] == "text"]
    discard(partial)                     # documented: treat as incomplete
```

The two retry shapes, chosen by the claim.

```
body = dict(request) | {"model": target,
                        "fallback_credit_token": token}
if claim is not False:
    echoed = [dict(b) for b in refused.content]
    if echoed and echoed[-1]["type"] == "text":
        echoed[-1]["text"] = echoed[-1]["text"].rstrip()
    body["messages"] = [*request["messages"],
                        {"role": "assistant", "content": echoed}]
```

Rejecting a guardrail before building it.

```
need = tolerable_specificity(base_rate=0.002, target_precision=0.5,
                             sensitivity=0.99)   # 0.998016
```

25.12 Code Walkthrough: The Reader That Assumes Nothing

The refusal reader is fourteen lines, and every defence in it is there because the documentation says the field it is guarding can be absent.

```
def read(payload):
    resp = wire.parse_response(payload)
    if resp.stop_reason != "refusal":
        return None
    d = resp.stop_details or {}
    partial = [b for b in resp.content if b.get("type") == "text"]
    return Refusal(category=d.get("category"),
                   explanation=d.get("explanation"),
                   credit_token=d.get("fallback_credit_token"),
                   prefill_claim=d.get("fallback_has_prefill_claim"),
                   partial_blocks=partial,
                   billed=bool(partial))
```

Line by line. The branch is on `stop_reason` and nothing else. The documentation's pitfall list says it twice — *"Branch on `stop_reason`, not on `stop_details` or `content`"* — and gives two independent reasons: `stop_details` is informational and can be `null` on a refusal, and on a Message Batch result it *"may be `null`"* as a matter of course.

`resp.stop_details or {}` turns the batch case into a lookup that returns `None` rather than an `AttributeError`. `d.get("category")` then returns `None` for two genuinely different situations that the API deliberately does not distinguish: a refusal with no named category, and a payload with no details at all. The code treats both as unnamed, which is the only honest reading available.

`partial` filters to text blocks rather than indexing. On a pre-output refusal `content` is `[]`; on a mid-output refusal it holds the text that already reached the user. `billed=bool(partial)` encodes the documented billing rule directly: a refusal before any output is not charged and does not count against rate limits; a mid-stream refusal bills the input and the output already streamed.

Compare with the two readers this project keeps in order to measure them:

```
def naive_read(payload):
    return payload["content"][0]["text"]      # raises on 6 of 7
def naive_monitor(payload, http_status=200):
    return http_status >= 400                 # sees 0 of 6
```

The second one is the more expensive bug, because it does not raise. It reports that everything is fine.

25.13 Mini Project: The Guardrail Ladder and the Fallback State Machine

What it does. Prices a classifier guardrail against a real base rate, reads every documented refusal shape, reproduces every documented fallback-configuration rejection offline, builds and walks the credit-redemption ladder, and ablates a nine-part prompt with paired intervals.

What it must not do. Make a network call, or use an unseeded random number.

```
  ch25/
  ├── guard.py          Classifier, screen(), stack(),
  │                     tolerable_specificity() — the base-rate maths
```

```
├── refusal.py        the refusal contract, fallback validation,
│                     build_retry(), the rejection ladder, credit
├── fixtures.py       2 DOC + 3 BUILT fixture groups, 7 refusals
├── ablate.py         leave-one-out prompt ablation with intervals
├── simulate_lite.py  the latent item model, copied from ch24/
├── report.py         every published figure
├── test_guard.py     33 tests
├── evalkit.py        copied from ch8/  — wilson, cohen_kappa
├── stats.py          copied from ch24/ — paired_interval, phi_inv
├── wire.py           copied from ch23/ — parse_response
└── schema.py, conform.py   copied from ch23/
```

```
cd ch25 && python3 -m pytest -q     # 33 passed
python3 report.py                   # every figure below
```

25.14 Panel A — What a 99/99 Guardrail Does to Real Users

```
base rate  5.00%  accuracy 99.00%  recall 99.0%  precision 83.90%
                                    0.19 legit blocked per catch
base rate  1.00%  accuracy 99.00%  recall 99.0%  precision 50.00%
                                    1.00 legit blocked per catch
base rate  0.20%  accuracy 99.00%  recall 99.0%  precision 16.56%
                                    5.04 legit blocked per catch
to reach 50% precision at a 0.2% base rate the layer needs
    specificity 0.998016 -- a false-positive rate of 0.1984%,
    5.0x better than 99%
two independent 90/99.5 layers in series: recall 99.0%,
    precision 16.59%, 9955 legitimate requests blocked
```

The accuracy column is constant at 99.00 per cent across a twenty-five-fold change in base rate, while precision falls from 83.90 to 16.56 and the human cost rises from one blocked customer per five catches to five blocked customers per catch. Chapter 8 said accuracy is a bad safety metric; this is the sentence with a price on it.

The stacking result is the one that surprises people. Two weaker layers in series reach the same 99.0 per cent recall as one strong layer and land at **16.59 per cent** precision with **9,955** legitimate requests blocked — essentially identical to the single layer. Composition multiplies the misses, which is the point, and it also compounds the false positives, which is not. **Layers in series buy recall and spend precision**, so a guardrail architecture that adds a layer whenever something slips through is walking down a curve nobody plotted.

The backwards calculation is the tool to take away. `tolerable_specificity(0.002, 0.50, 0.99)` returns **0.998016** in a millisecond, and it is the number that should appear in the design document beside the classifier's measured false-positive rate and its Wilson interval.

25.15 Panel B — Seven Refusals, One Reader

```
id       category              content  billed  credit  claim
tvVoYEL  cyber                       0   False   False   None
msg_b01  cyber                       0   False    True  False
msg_b02  bio                         1    True    True   True
```

```
msg_b03  frontier_llm                0   False    True  False
msg_b04  reasoning_extraction        0   False   False   None
msg_b05  None                        0   False    True  False
msg_b06  None                        0   False   False   None
content[0].text raises on 6 of 7
an error-rate monitor sees 0 of 6 refusals
4 of 6 carry a named category; the rest are permanently null
```

Seven payloads: the documentation's own example plus one BUILT fixture per documented category, one with a `null` category, and one with `stop_details` absent entirely as a batch result may be.

`content[0].text` **raises on six of them.** Chapter 16 named this bug when it raised on three of seven stop reasons. Refusals make it worse rather than better: `content` is not the wrong block, it is `[]`.

An error-rate monitor sees zero of six. The documentation says it in one line and it is the most operationally important line on the page: *"A refusal is an HTTP 200, so monitoring built on error rates or 5xx responses never sees it. Emit one event per refusal and one per fallback-served response, then alert on the gap between the two counts."* That gap is the number that matters — refusals that were **not** rescued by a fallback are the ones your users experienced.

Exactly one fixture is billed. The mid-output `bio` refusal, which produced 206 characters and 48 output tokens before the classifier fired. The other six produced nothing and cost nothing, and did not count against rate limits either.

Two of six carry no category. Both are readable without special-casing because the reader never assumes the field exists.

25.16 Panel C — Eight Fallback Rejections, No API Key

```
CAUGHT  no beta header                          FallbackError
CAUGHT  on the Message Batches API              FallbackError
CAUGHT  on Amazon Bedrock                       FallbackError
CAUGHT  four fallback entries                   FallbackError
CAUGHT  duplicate entries                       FallbackError
CAUGHT  fallback equals the requested model     FallbackError
CAUGHT  a target that is not permitted          FallbackError
CAUGHT  a model with no published target list   FallbackError
8 of 8 caught with no API key
DOC fallback response: model field reports claude-opus-4-8,
    first content block is 'fallback'
usage.iterations has 2 entries, 1 of them fallback_message
```

The last case is Chapter 14's rule again in a new place. `claude-sonnet-5` has no published `allowed_fallback_models` list in this project's records, so the validator refuses rather than guessing a plausible target. The documentation's instruction is to read the list from the Models API with the beta header set; a hard-coded guess is exactly the kind of fact that is right until it is not.

The DOC fallback response confirms two things a parser must handle. The top-level `model` field reports **the model that answered**, not the one you asked for — so any code that asserts `response.model == request.model` breaks the first time a fallback fires. And the **first content block is a** `fallback` **block**, which means `content[0]` is now wrong in a third distinct way.

25.17 Panel D — The Retry, and What the Credit Is Worth

```
prefill_claim True  -> retry appends an assistant message: True
echoed text is rstripped: True
prefill_claim False -> unchanged body: True
after 0 rejection(s): continuation body
after 1 rejection(s): unchanged body + token
after 2 rejection(s): unchanged body, no token
  4,000-token prefix on Opus 4.8: $0.0250 -> $0.0020  (12.5x)
 40,000-token prefix on Opus 4.8: $0.2500 -> $0.0200  (12.5x)
200,000-token prefix on Opus 4.8: $1.2500 -> $0.1000  (12.5x)
documented check: delta_write -40000, delta_read +40000,
    applied True
```

The **12.5×** is exact and is the same at every prefix size, because it is the ratio of two constants: a five-minute cache write at 1.25× against a read at 0.1×. Chapter 19 established that those multipliers are constant ratios rather than prices; this is what that buys you. On a 200,000-token agent context — entirely ordinary in Part 9 — a single refused turn costs **$1.25** to re-warm without the credit and **$0.10** with it.

The rejection ladder does what the documentation describes and no more. Note what the third rung silently drops: `fallback_credit_token` is absent from the final body, because the credit has been forfeited. That is the correct behaviour and it is also the rung with the trap — if the refused request executed server tools, a tokenless retry **re-runs and re-bills them**, and the documented instruction is to surface the 400 to your caller instead of taking this rung at all. A generic retry helper that walks the ladder blindly will re-run a web search and a code execution container, and the bill will arrive in a different line item from the one anybody is watching.

The verification check is the documented one: writes fall by exactly the amount reads rise. A shift of zero is not a failure — it means the token was honoured and the fallback model's cache was already warm.

25.18 Panel E — The Tokens You Have Already Shown Someone

```
mid-output refusal: 206 characters, 48 output tokens, already
    streamed and already billed
the documentation's instruction: treat any partial output as
    incomplete and discard it
ends: ...'ed with standard equipment, and then to '
```

Chapter 17's rule was **render freely, but act only after** `message_stop`. This is the case that shows why the second half is not pedantry. Two hundred and six characters of a plausible, helpful-sounding answer have already been painted into somebody's browser, and the correct action is to remove them. There is no version of this that is not visible to the user; the choice is between removing the text and explaining, or leaving text on screen that the platform has told you is incomplete.

Non-streaming makes the choice for you by discarding the partial output server-side and answering from scratch — while still reporting the declined attempt's output tokens in `usage.iterations`. So the same decline is, in the streaming path, a continuation the user watches happen, and in the non-streaming path a clean answer with an invisible extra charge.

25.19 Panel F — Ablation, One Part at a Time

```
  role              +0.30 pts   95% CI [ -1.21,  +1.81]
* task              +7.20 pts   95% CI [ +5.50,  +8.90]
* reasoning_rule    +3.00 pts   95% CI [ +1.39,  +4.61]
  format_rule       -1.40 pts   95% CI [ -2.81,  +0.01]
* examples          +3.90 pts   95% CI [ +2.28,  +5.52]
  tone_rule         -0.60 pts   95% CI [ -2.14,  +0.94]
* anti_laziness     -2.65 pts   95% CI [ -4.10,  -1.20]
  self_verify       +0.50 pts   95% CI [ -1.02,  +2.02]
  closing_reminder  +0.35 pts   95% CI [ -1.11,  +1.81]
 4 of 9 parts have an established effect; 5 do not
 1 part is established as HARMFUL: anti_laziness at -2.65 points
```

Nine parts, 2,000 paired cases each, leave-one-out. **Four have an established effect and five cannot be shown to do anything.** That is the ordinary condition of a real prompt: it accretes, exactly as Chapter 21 measured for system prompts, and most of what accretes is inert.

The starred negative is the interesting one. `anti_laziness` **is established as harmful at −2.65 points**, which is Chapter 20's documented finding arriving as a measurement: anti-laziness prompting now over-triggers on current models, and *"If in doubt, use [tool]"* is called out by name in the documentation. A team that added that line in 2024 and never removed it is paying 2.65 points for it.

Note also `format_rule` at −1.40 with an interval of [−2.81, +0.01] — a hair from established, and the right response is *more data*, not a decision. Chapter 24's discipline applies to your own debugging as much as to somebody else's benchmark.

```
baseline               88.60%
drop anti_laziness     90.80%
drop examples          84.15%
drop both              85.15%
changing two things at once reads -3.45 points and hides a
    +2.20-point improvement
```

And there is the failure that motivated the section. Removing the anti-laziness instruction is worth **+2.20 points**. Removing the examples costs **−4.45**. Remove both and the meter reads **−3.45**, from which a reasonable engineer concludes that the anti-laziness instruction was helping and puts it back. Two edits, one measurement, one confident and completely wrong causal story.

25.20 Evaluation and Deployment

Instrument refusals as a first-class signal. One event per refusal, one per fallback-served response, and an alert on the **gap** between them. No 4xx will ever fire.

Put fallback on every request path. The documentation names the paths people miss: retry handlers, error-recovery branches, background workers and sub-agent calls made inside tool execution. *"A handler that re-issues a request without fallback loses the protection on exactly the requests most likely to need it."*

Make fallback a property of the request. Not a shared flag, not a cached config value, not a global toggle. *"When you cannot confirm fallback is active, configure it rather than assume it is on."*

Budget retries per request, not per turn or per session. One turn can produce several refusals — an agent plus its sub-agents.

Roll a guardrail out in shadow mode first. Log what it would have blocked, against real traffic, for long enough to estimate the false-positive rate with an interval. The base rate you assumed at design time is almost never the one you have.

25.21 Hands-on Labs

Lab 1 — Price your guardrail. Take a classifier you run and compute its precision at your real base rate. **Success criterion:** a number for legitimate requests blocked per genuine catch.

Lab 2 — Break your refusal handler. Feed it a payload with `content: []` and one with `stop_details: null`. **Success criterion:** neither raises, and both emit a metric.

Lab 3 — Validate a fallback configuration offline. Reproduce at least five of the eight documented rejections. **Success criterion:** five distinct errors, no API key.

Lab 4 — Compute your credit. Take your largest cached prefix and price the retry with and without the token. **Success criterion:** a figure, and the platform list you are allowed to use it on.

Lab 5 — Ablate a real prompt. Split one of your prompts into parts and run leave-one-out with paired intervals. **Success criterion:** a list of parts you cannot show do anything.

25.22 Lab Solutions

1. `guard.screen(clf, base_rate)`. If you do not know your base rate, that is the finding; shadow-mode logging gives it to you in a fortnight. Then run `tolerable_specificity` backwards and compare with the classifier's measured false-positive rate.

2. The reader must use `resp.stop_details or {}` and `.get()` throughout, and must filter `content` by type rather than indexing. Assert that a `null` category is passed through as an `unnamed` metric tag rather than dropped.

3. `refusal.validate_fallbacks` with the beta header omitted, with `batch=True`, on a non-API platform, with four entries, with duplicates, and with a target that is not in `allowed_fallback_models`.

4. `refusal.credit_saving(prefix, price_in)`. The ratio is always 12.5 because both multipliers are constants; only the absolute saving scales. Remember that credit reaches Bedrock, Google Cloud and Microsoft Foundry while server-side fallback does not.

5. `ablate.run()` is the template. The essential parts are that each comparison is **paired** on identical cases and that each reports an interval; without both you are reading noise. Expect roughly half your parts to be inert.

25.23 Best Practices

BEST PRACTICE

Compute precision at your real base rate before building a guardrail

Accuracy is constant across base rates and tells you nothing. Precision and blocked-users-per-catch are the numbers with consequences.

BEST PRACTICE

Route, do not block, below about 99.8 per cent specificity

A flagged request can go to a slower, more careful path, or ask a clarifying question. Blocking is the highest-cost action available and should need the highest confidence.

BEST PRACTICE

Branch on `stop_reason`, never on `stop_details` or `content`

`stop_details` can be `null` on a refusal and is routinely `null` on batch results; `content` can be the empty array.

BEST PRACTICE

Emit a refusal metric and a fallback-served metric, and alert on the gap

No error-rate monitor will ever see a refusal. The gap between the two counts is the number of users who got nothing.

BEST PRACTICE

Retry on a different model

Re-sending a refused request to the same model usually earns another refusal.

BEST PRACTICE

Configure fallback on every request path, including sub-agents

`fallbacks` does not propagate into model calls made inside tool execution, and the paths that re-issue requests are exactly the ones that need it.

BEST PRACTICE

Make fallback a property of the request, not of ambient state

A shared flag can drift. When you cannot confirm it is active, set it.

BEST PRACTICE

Read `allowed_fallback_models` from the Models API

A hard-coded target list is a fact with no verification date.

BEST PRACTICE

Redeem the credit token, and check that it applied

Writes should fall by exactly the amount reads rise. Twelve and a half times, on every prefix.

BEST PRACTICE

Do not take the tokenless rung when server tools have run

It re-runs and re-bills them. Surface the 400 instead.

BEST PRACTICE

Discard partial output on a refusal

The documentation is unambiguous: treat it as incomplete. Render freely, act after `message_stop`.

BEST PRACTICE

Validate in three senses, and build the third one first

Shape, sense, authority. Only the authority check prevents harm, and only it is evaluated against the caller rather than the model's request.

BEST PRACTICE

Triage before you edit a prompt

Chapter 10's four questions, then `stop_reason`, then configuration, then words. Most prompt debugging sessions should not have started.

BEST PRACTICE

One change per measurement

Two edits and one number is a confident wrong story, measured here at −3.45 points concealing +2.20.

BEST PRACTICE

Delete the prompt parts you cannot show do anything

Five of nine, in this project. They cost tokens, they cost cache stability, and one of them is costing 2.65 points.

25.24 Common Mistakes

COMMON MISTAKE

Shipping a classifier measured on a balanced set

Symptom. Excellent offline numbers, a support queue full of blocked customers.

Cause. Precision depends on the base rate; accuracy does not. At 0.2 per cent, a 99/99 classifier has 16.56 per cent precision.

Fix. Measure on realistically imbalanced traffic, report precision with a Wilson interval, and run `tolerable_specificity` backwards before building.

Detection. 5.04 legitimate users blocked per genuine catch, computed in one line.

COMMON MISTAKE

Alerting on error rates

Symptom. A clean dashboard during a refusal spike.

Cause. A refusal is an HTTP 200.

Fix. Dedicated refusal and fallback-served counters, alerting on the gap.

Detection. An error-rate monitor sees 0 of 6 refusal fixtures.

COMMON MISTAKE

`content[0].text` on a refusal

Symptom. `IndexError` in the safety path, which is the worst possible place for an unhandled exception.

Cause. `content` is `[]` on a pre-output refusal, and the first block is a `fallback` block on a fallback-served response.

Fix. Filter blocks by type. Never index.

Detection. Raises on 6 of 7 fixtures here.

COMMON MISTAKE

Treating fallback as a resilience feature

Symptom. A rate-limit storm with no fallback in sight.

Cause. *"Only a safety classifier decline triggers the fallback. A rate limit, overload, or server error on the requested model is returned to you as-is."*

Fix. Keep Chapter 19's retry and backoff discipline entirely separately.

Detection. Count fallback-served responses by cause; they should all be refusals.

COMMON MISTAKE

Asserting the response model equals the requested model

Symptom. A hard failure the first time a fallback fires in production.

Cause. The top-level `model` field reports the model that answered.

Fix. Read the serving model from `model`, the boundary from the `fallback` block's `to.model`, and the accounting from `usage.iterations`.

Detection. The DOC fixture reports `claude-opus-4-8` for a request that named `claude-fable-5`.

COMMON MISTAKE

Moving or dropping the `fallback` block when echoing a turn

Symptom. A 400 on the next turn of a conversation that just worked.

Cause. The API uses the block's position to validate the thinking blocks around it.

Fix. Keep it exactly where it appeared; drop `thinking`, `redacted_thinking`, `connector_text` and client `tool_use` that precede it.

Detection. A round-trip test on a fallback-served turn, as Chapter 18 did for thinking.

COMMON MISTAKE

Falling through to a tokenless retry after server tools ran

Symptom. A duplicated web-search or code-execution bill nobody can explain.

Cause. The last rung of the rejection ladder re-runs completed server tools.

Fix. Surface the 400 to the caller when server tools executed.

Detection. Compare server-tool invocation counts with request counts.

COMMON MISTAKE

Assuming the credit works everywhere fallback does

Symptom. A retry that quietly pays the write price.

Cause. Server-side fallback is Claude API and Claude Platform on AWS; fallback credit additionally covers Bedrock, Google Cloud and Microsoft Foundry. Batch tokens cannot be redeemed at all.

Fix. Check the platform, and check the shift in `cache_creation_input_tokens`.

Detection. `credit_applied()` returns `applied: False` with a positive write delta.

COMMON MISTAKE

Leaving partial output on the screen after a refusal

Symptom. A user acting on half an answer the platform declined to finish.

Cause. The mid-stream text was already rendered.

Fix. Discard it and say so. Design the interface so that removing text is possible.

Detection. 206 characters, in this project's fixture.

COMMON MISTAKE

Debugging by editing

Symptom. A week of prompt changes and no attributable effect.

Cause. No ablation, no pairing, no intervals, and often two changes at a time.

Fix. Leave-one-out with paired intervals on a set with power.

Detection. Four of nine parts established; five not; one harmful; and a two-at-once experiment reading −3.45 where the truth was +2.20.

25.25 Performance Considerations

Server-side fallback costs a round trip in latency, not in requests. On a decline before any output, "`message_start` *waits for the fallback attempt to start, [so] time to first byte includes the declined attempt.*" You have spent the first model's prefill and got nothing for it. Chapter 11's two-term model applies twice.

A guardrail is a serial latency cost. An input classifier on the critical path adds its full latency to every request, including the 99.8 per cent that are fine. Run it concurrently with the main call and cancel on a flag, or move it to the action layer where it runs once per effect rather than once per token.

The credit is a cost lever, not a latency one. It reprices the prefix; it does not avoid re-processing it.

Ablation is embarrassingly parallel and belongs in Batch. Nine parts times 2,000 cases is 18,000 calls with no latency requirement, at the documented 50 per cent discount and on a separate rate-limit bucket (Chapter 19).

Refusals that arrive before output do not count against rate limits. A refusal storm therefore does not eat your RPM, which is a small mercy and a reason your rate-limit dashboard will also be quiet during one.

25.26 Security Considerations

A guardrail you can see is a guardrail you can tune against. An input classifier that returns a distinctive message tells an attacker exactly which phrasings trip it, turning your filter into an oracle. Return the same user-facing response for a block as for a benign decline, and keep the detail in your logs.

The explanation string is not for parsing and not for users. It is *"not stable"*, and it names the policy area — which is information about the classifier. Display it if it helps, log it always, branch on it never.

The action layer must authorise against the caller, not the request. The most dangerous response in this chapter is not the refusal; it is the schema-conformant, entirely plausible instruction to move money that arrived because a document said so. Chapter 21 gave the principle, Chapter 23 gave the shape, and Chapter 58 gives the attack.

A refusal is a signal about your traffic. A spike in `frontier_llm` or `cyber` refusals is worth investigating as a security event, not just as an availability one — and it will be invisible if you are watching error rates.

Credit tokens are bearer tokens. They are opaque, they are scoped, they expire in five minutes, and they should be treated like any short-lived credential: never logged in full, never returned to a client,

never persisted.

25.27 Real Production Examples

A developer-tools company ran a jailbreak filter at 99/99 in front of a code assistant. The genuine attack rate was 0.2 per cent. After two weeks the filter had blocked roughly five thousand legitimate requests per million and stopped fewer than two thousand attacks; it was moved from blocking to routing — flagged requests went to a higher-effort model with a stricter system prompt — and complaints stopped without measurably changing what got through.

A life-sciences platform built on Fable 5 and found that a fraction of entirely legitimate protocol questions came back as `bio` refusals, exactly as the documentation warns. Server-side fallback to Opus 4.8 fixed the user experience in one line. Their write-up's most useful sentence was that they only noticed at all because they had added a refusal counter; the error-rate dashboard had been green throughout.

An agent platform implemented the credit ladder generically and re-ran a code-execution container on every third rung. The fix was four lines: detect that server tools had executed and surface the 400 instead. The bug had been invisible because the container charge lands in a different line item from the model charge.

A support product spent three weeks tuning a prompt that had accreted nine instructions over two years. The ablation took an afternoon: four parts mattered, five did not, and the anti-laziness line added in 2024 was costing 2.65 points. Deleting five parts also shortened the cached prefix, which was the second-largest cost saving of the quarter.

25.28 Certification Notes

Concept	Domain	Weight	What is examinable
Layered guardrails	D5 Context Management & Reliability	15%	which layer can prevent harm
Base rate and precision	D5	15%	accuracy is constant; precision is not
The refusal contract	D4 Prompt Engineering & Structured Output	20%	HTTP 200, empty `content`, nullable `stop_details`
The four refusal categories	D4	20%	`cyber`, `bio`, `frontier_llm`, `reasoning_extraction`
Refusal billing	D5	15%	pre-output refusals are not billed and do not count against rate limits
Server-side fallback	D1 Agentic Architecture & Orchestration	27%	beta header, three entries, permitted targets, not on Batch
The `fallback` block and `usage.iterations`	D1	27%	the serving model is the top-level `model` field

Concept	Domain	Weight	What is examinable
Fallback credit	D5	15%	the token, the prefill claim, the rejection ladder
Validation in three senses	D5	15%	authority is evaluated against the caller
Ablation with intervals	D4	20%	one change per measurement

CERTIFICATION NOTE

Exam tips

The refusal facts are the highest-yield material in this chapter. Memorise that a refusal is an **HTTP 200**, that `content` may be **empty**, that `stop_details` may be **null**, and that the four categories are `cyber`, `bio`, `frontier_llm` and `reasoning_extraction`.

Billing is a favourite: **not billed before any output; billed for what streamed on a mid-stream refusal.**

If a question says a model was overloaded or rate-limited and asks whether fallback fires, the answer is **no** — only a safety classifier decline triggers it.

If a question involves a retry across models and a large cached prefix, the answer involves **fallback credit**.

And if a question asks why a 99 per cent accurate filter caused an incident, the answer is the **base rate**.

25.29 Twenty Practice Questions

1. Name the four guardrail layers and say which can prevent harm.
2. A 99/99 classifier at a 0.2 per cent base rate: what is its precision?
3. Why does accuracy not move as the base rate changes?
4. What specificity is needed for 50 per cent precision at a 0.2 per cent base rate with 99 per cent sensitivity?
5. What do two layers in series buy, and what do they spend?
6. What HTTP status accompanies a refusal?
7. What may `content` contain on a pre-output refusal?
8. Name the four documented refusal categories.
9. When are `category` and `explanation` null, and is that permanent?
10. When is a refusal billed?
11. What must you branch on, and what must you not branch on?
12. Name the three fallback approaches and when each applies.
13. What beta header does server-side fallback require, and how exact must it be?
14. How many fallback entries may you name?
15. What conditions trigger a server-side fallback, and what conditions do not?
16. Where does the response report the model that answered?
17. What does `fallback_has_prefill_claim` decide?

18. Give the three rungs of the credit rejection ladder.
19. When must you not take the last rung?
20. Why is one change per measurement not merely good hygiene?

25.30 Answers and Explanations

1. **Input, model, output, action** — only the **action** layer prevents harm, because only it sits between the model and the effect. *Distractor:* the model layer refuses, but it cannot be relied on alone and cannot be configured.
2. **16.56 per cent.** *Distractor:* 99 per cent is the accuracy, which is the trap.
3. Because accuracy is dominated by the true negatives, which are almost everything when the base rate is low. *Distractor:* "the classifier changes" — it does not; only the population does.
4. **0.998016**, a false-positive rate of 0.1984 per cent — five times better than 99 per cent.
5. They buy **recall** (misses multiply down) and spend **precision** (false positives compound). Two 90/99.5 layers give 99.0 per cent recall and 16.59 per cent precision.
6. **200.**
7. **The empty array.** *Distractor:* "a text block with a refusal message" is what people assume and is why `content[0].text` raises.
8. `cyber`, `bio`, `frontier_llm`, `reasoning_extraction`.
9. When the refusal maps to no named category. It is *"a normal, permanent value, not a placeholder."*
10. **Not at all if it arrives before any output** — and it does not count against rate limits. A **mid-stream** refusal bills input plus the output already streamed.
11. Branch on `stop_reason`; not on `stop_details` (nullable, and routinely null on batch results) and not on `content` (may be empty).
12. **Server-side fallback** on the Claude API and Claude Platform on AWS; **the SDK middleware** on any platform with the TypeScript, Python, Go, Java or C# SDK; **a manual retry with fallback credit** on Ruby, PHP, raw HTTP or custom logic.
13. `server-side-fallback-2026-06-01`, and it must carry exactly that date; any other `server-side-fallback-*` value is a 400.
14. **Up to three**, distinct from each other and from the requested model, each a permitted target.
15. **Only a safety classifier decline.** Rate limits, overload and server errors are returned as-is.
16. The **top-level** `model` **field**; the boundary is in the `fallback` block's `to.model` and the accounting in `usage.iterations`.
17. Which retry body to send: `true` means append an assistant message echoing the refused content so the retry **continues** and completed server tool calls are not re-executed; `false` means send the unchanged body.
18. **Continuation body → unchanged body with the token → unchanged body without the token** (credit forfeited).
19. When the refused request **executed server tools** — a tokenless retry re-runs and re-bills them; surface the 400 instead.

20. Because two edits and one measurement produce an attributable-looking number that is not attributable. Measured here: −3.45 points concealing a +2.20-point improvement.

25.31 Ten Practical Exercises

1. Compute your guardrail's precision at your real base rate.
2. Add a refusal counter and a fallback-served counter, and alert on the gap.
3. Feed your handler a `content: []` payload and a `stop_details: null` payload.
4. Add `fallbacks` to every request path, including sub-agent calls.
5. Read `allowed_fallback_models` from the Models API and check it into your registry with a date.
6. Price your largest prefix with and without the credit.
7. Add a guard that refuses the tokenless rung when server tools ran.
8. Make your streaming UI capable of removing rendered text.
9. Split one prompt into parts and ablate it.
10. Delete every part you could not show does anything, and re-measure.

25.32 Five Challenge Labs

1. **Shadow-mode guardrail.** Run a classifier against live traffic without blocking, estimate its false-positive rate with a Wilson interval, and decide from data whether to promote it to blocking, routing, or the bin.
2. **The fallback state machine.** Implement all three approaches behind one interface, with a property test that the manual path and the server-side path produce equivalent conversation histories.
3. **The credit auditor.** Verify on every retry that writes fell by the amount reads rose, and alert when a token is silently not applied.
4. **The refusal taxonomy dashboard.** Break refusals down by category over time and correlate `cyber` and `frontier_llm` spikes with traffic sources.
5. **The automatic ablator.** Given a prompt split into parts and an eval set, run leave-one-out nightly and open a pull request deleting any part whose interval has contained zero for a month.

25.33 Ten Interview Questions

1. Your filter is 99 per cent accurate and support is furious. Explain.
2. Where would you put the only guardrail you were allowed to build?
3. Why does an error-rate dashboard stay green during a refusal spike?
4. What does `content[0].text` do on a refusal, and why does it matter here more than elsewhere?
5. A model was overloaded. Does fallback fire?
6. Walk me through a manual retry with fallback credit.
7. When would you not forfeit the token and retry anyway?
8. A mid-stream refusal has already rendered 200 characters. What now?

9. Half your prompt does nothing. How would you find out which half?
10. Your colleague changed two things and the number moved. What do you say?

25.34 Summary

Part 4 has been about making a model do what you want: principles, the system prompt, structure, schemas, evaluation. This chapter is about the other half of production, which is what your system does when the model says no, when the output is wrong in a way no schema can see, and when the prompt you inherited contains nine instructions of which five do nothing.

Guardrails are layers at four points — input, model, output and action — and only the last one prevents harm, because only it stands between the model and the effect. Every guardrail you write is a classifier, and a classifier's consequences are set by a number that has nothing to do with it: the base rate of the thing it is looking for. One filter at 99 per cent sensitivity and 99 per cent specificity has 83.90 per cent precision at a five per cent base rate and **16.56 per cent** at 0.2 per cent, blocking **five legitimate users for every genuine catch**, with the accuracy figure sitting unchanged at 99.00 per cent throughout. Reaching even 50 per cent precision at that base rate would demand a specificity of **0.998016**, and stacking two weaker layers in series buys recall while compounding the false positives — 16.59 per cent precision and 9,955 blocked requests, for twice the latency.

The refusal contract is small and every part of it breaks a naive handler. A refusal is an **HTTP 200** whose `content` may be the **empty array** and whose `stop_details` may be **null**, permanently and by design. Four categories are documented — `cyber`, `bio`, `frontier_llm` and `reasoning_extraction` — and three of them come with an explicit acknowledgement that legitimate work triggers them. A refusal before any output is **not billed and does not count against rate limits**; a mid-stream refusal bills the input and the text your user has already read. Across seven fixtures, `content[0].text` raises on six and an error-rate monitor sees **zero of six**, which is why the documentation's instruction is to emit a refusal metric and a fallback-served metric and alert on the gap between them.

Falling back has three documented shapes. Server-side `fallbacks` — up to three permitted targets, an exact beta date, not on Batch and not on the three third-party clouds — turns a decline into one round trip, marks the handoff with a `fallback` content block, reports the serving model in the top-level `model` field, and records every attempt in `usage.iterations`. The SDK middleware does the same on any platform. And a manual retry needs **fallback credit**: an opaque token on the refusal, a Boolean that chooses between continuing the partial output and starting over, and a repricing worth exactly **12.5×** on the prefix because a cache write at 1.25× against a read at 0.1× is a constant. The rejection ladder has three rungs and the last one is a trap: when server tools have already run, a tokenless retry re-runs and re-bills them, and the documented advice is to surface the error instead.

Debugging closes the part, and it is the least glamorous section in the book on purpose. Triage before editing; check `stop_reason` before content and configuration before words; then ablate one part at a time with paired intervals on a set that has power. On a nine-part prompt, **four parts have an established effect and five cannot be shown to do anything at all** — and one, the anti-laziness instruction the documentation now warns about by name, is established as **harmful at −2.65 points**. A team that removed it together with the examples read **−3.45** and concluded the opposite of the truth. That is the shape of most prompt debugging: not a failure of insight, a failure of experimental design, and Chapter 24's arithmetic is the whole cure.

KEY TAKEAWAYS

1. **Only the action layer prevents harm.** Input filters reduce load and output checks reduce embarrassment; authorisation against the caller's identity is the one that stops the effect.
2. **A guardrail's consequences are set by the base rate, not by its accuracy.** 99/99 at 0.2 per cent means 16.56 per cent precision and five blocked customers per catch.
3. **Layers in series buy recall and spend precision.** Adding a layer per incident walks down a curve nobody plotted.
4. **A refusal is an HTTP 200 with possibly empty `content` and possibly null `stop_details`.** Branch on `stop_reason` alone.
5. **Four categories:** `cyber`, `bio`, `frontier_llm`, `reasoning_extraction` — and three of them are documented as catching legitimate work.
6. **A pre-output refusal is free; a mid-stream refusal bills what you already showed the user.** Discard the partial output regardless.
7. **No error-rate monitor will ever see a refusal.** Count refusals and fallback-served responses separately and alert on the gap.
8. **Only a safety decline triggers fallback.** Rate limits, overload and server errors remain entirely your problem.
9. **Fallback credit reprices the prefix at exactly 12.5×,** and the last rung of the rejection ladder re-runs server tools.
10. **Debugging is an ablation, not an edit session.** One change per measurement, paired, with intervals — or the number you get will be attributable to nothing.

25.35 Glossary

Term	Definition
Guardrail	A check at one of four layers — input, model, output, action — that constrains what a system will do.
Base rate	How often the thing a classifier looks for actually occurs; it sets precision and does not affect accuracy.
Precision	Of the requests flagged, the share genuinely harmful.
Recall / sensitivity	Of the genuinely harmful requests, the share flagged.
Specificity	Of the genuinely benign requests, the share passed.
Refusal	A successful HTTP 200 response with `stop_reason: "refusal"`.
`stop_details`	The object on a refusal carrying `type`, `category` and `explanation`; null for every other stop reason and possibly null on batch results.
Refusal category	One of `cyber`, `bio`, `frontier_llm`, `reasoning_extraction`, or null.
Server-side fallback	The `fallbacks` parameter with `server-side-fallback-2026-06-01`; retries a declined request inside one API call.

Term	Definition
`fallback` content block	`{"type": "fallback", "from": {...}, "to": {...}}`, marking a model handoff; in a stream, a start/stop pair with no deltas.
`usage.iterations`	The per-attempt accounting array; declined attempts are `message`, the serving attempt is `fallback_message`.
Fallback credit	A token on the refusal that reprices the retry's prefix as a cache read rather than a write.
`fallback_has_prefill_claim`	The Boolean that chooses between a continuation retry and an unchanged-body retry.
Rejection ladder	Continuation body → unchanged body with token → unchanged body without token.
Shape / sense / authority	The three independent senses in which an output must be validated.
Ablation	Leave-one-out measurement of a prompt's parts, paired and with intervals.

25.36 Further Reading

Primary sources, read on 1 August 2026. `platform.claude.com/docs/en/build-with-claude/refusals-and-fallback` is the authority for the refusal shape, the four categories, the billing rule, all three fallback approaches, the `fallback` block, `usage.iterations`, the echo rules, the streaming and non-streaming asymmetry, the batch behaviour, and the seven common pitfalls quoted throughout this chapter. `platform.claude.com/docs/en/build-with-claude/fallback-credit` covers the token, `fallback_has_prefill_claim`, the rejection ladder, the five-minute window, the platform list and the verification check. `platform.claude.com/docs/en/build-with-claude/handling-stop-reasons` is the full stop-reason reference. The guardrail pages under `platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/` — reduce hallucinations, increase output consistency, mitigate jailbreaks, reduce prompt leak, and handle streaming refusals — are the documented prompt-level techniques that complement the architectural ones here.

Where to go next in this book. Chapter 26 opens Part 5 with authentication, SDKs and REST, and takes the API surface from the outside in. **Chapter 29** is errors, retries, timeouts and backoff — the resilience half that fallback explicitly does not cover. **Chapter 31** onwards is tool use, where the action layer of Section 25.7 becomes a concrete design problem. **Chapter 56** catalogues reliability failure modes in the round and **Chapter 57** builds the observability this chapter's metrics belong in. **Chapter 58** is prompt injection and least privilege — the attack that the three-sense validation of Section 25.7 exists to survive.